ONNX June Meetup 2026

ONNX AND THE FUTURE OF VERIFIABLE AI

As AI becomes infrastructure, reproducibility and verification may become as important as model quality.

Presented by Sean Westfall

OBJECTIVE

RESEARCH DIRECTION / EXPLORATORY PROPOSAL

the objective of the presentation is to present a technical solution to a problem in zkML (zero knowledge machine learning), specifically for deterministic inference.

Part 1: introduction to concepts, Part 2: technical explanation

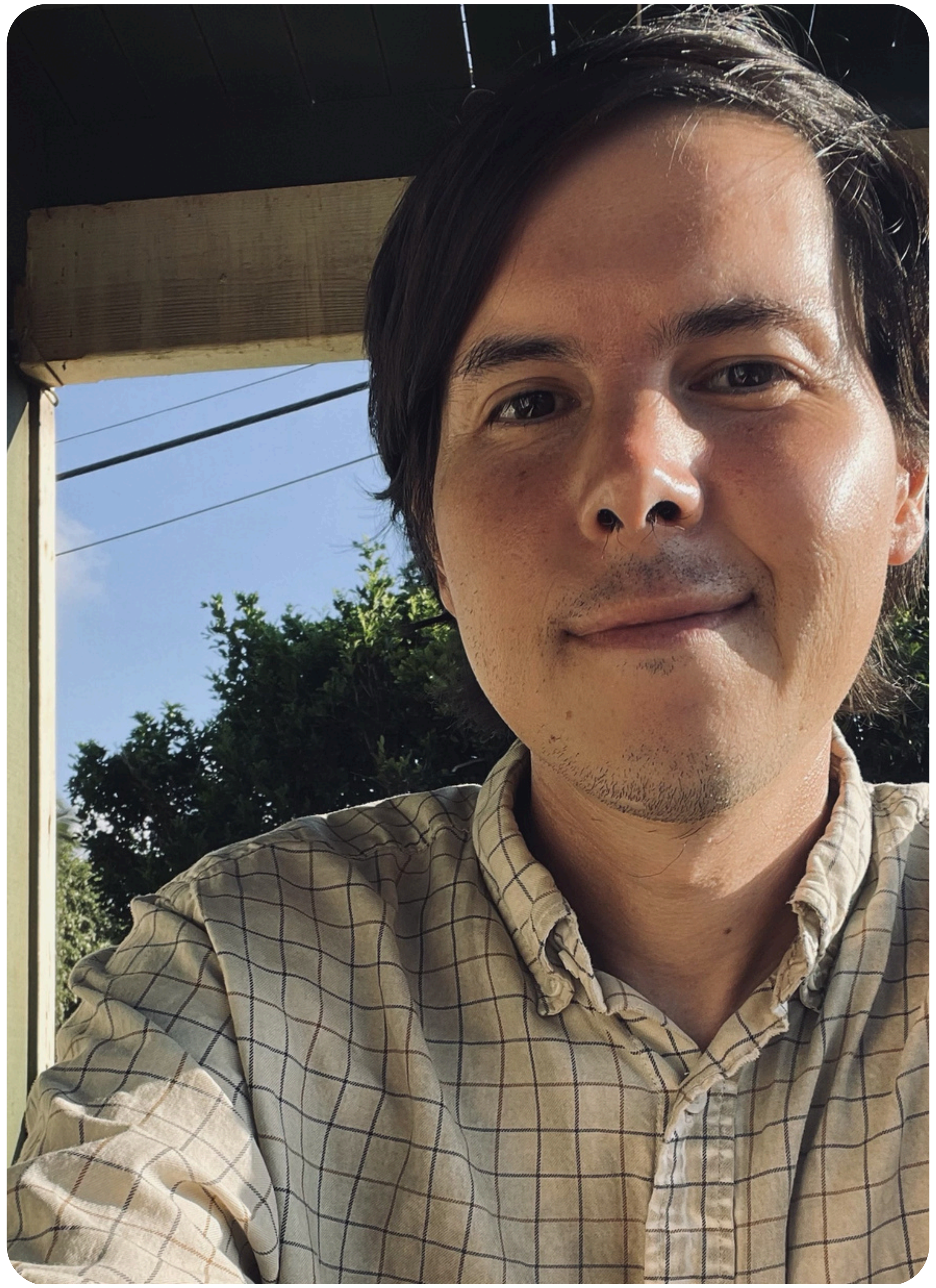
looking for sanity check before investing more time into this concept.

ABOUT ME

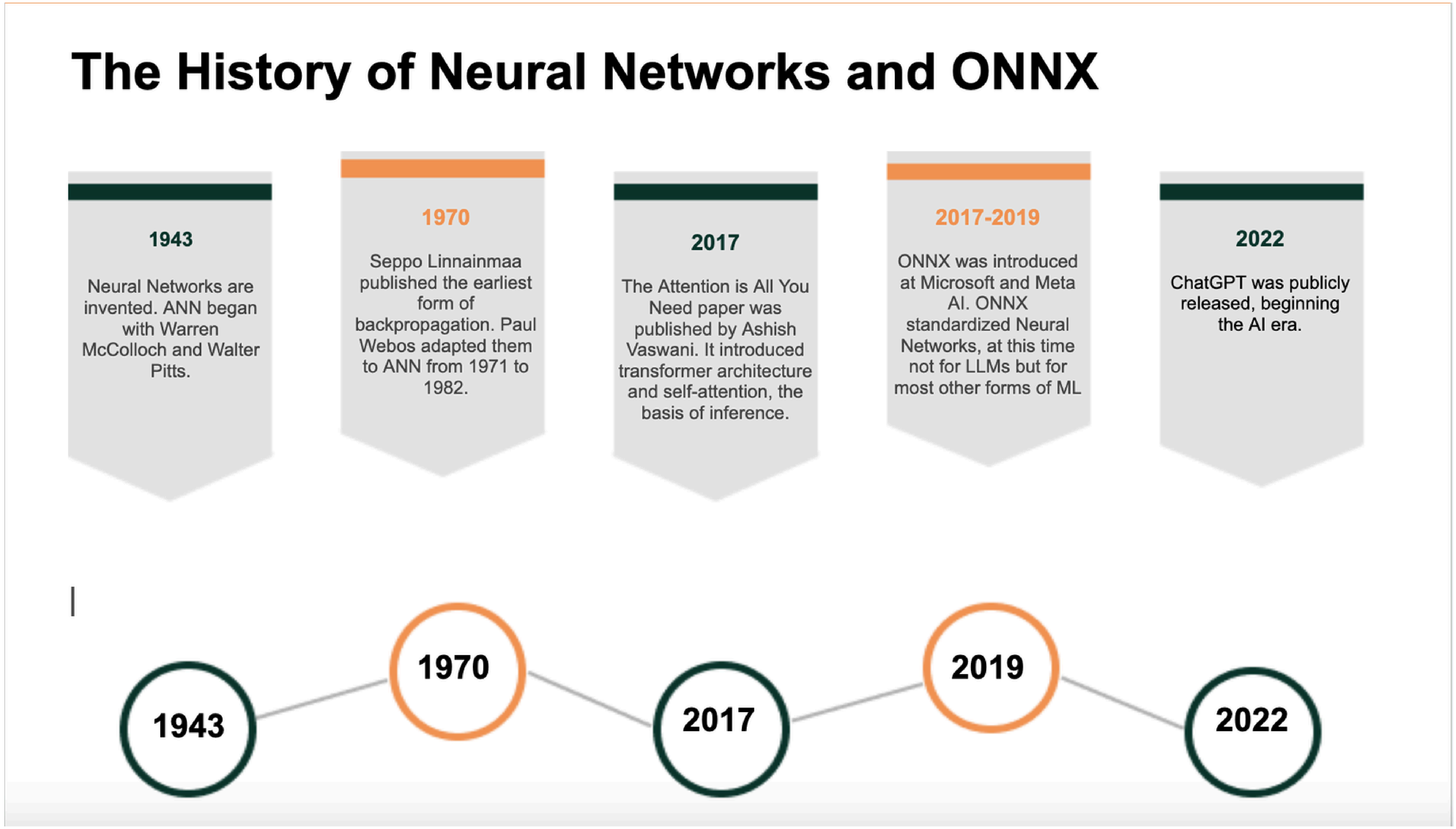
Hi 👋 I’m Sean Westfall 🇺🇸

- CS background
- consulting / systems integration
- interest in verifiable computing and AI

The last public presentation I did was on typechecking for statically typed programming languages.



WHAT IS ONNX


The History of Neural Networks and ONNX
1943
Neural Networks are invented. ANN began with Warren McColloch and Walter Pitts.
1970
Seppo Linnainmaa published the earliest form of backpropagation. Paul Webos adapted them to ANN from 1971 to 1982.
2017
The Attention is All You Need paper was published by Ashish Vaswani. It introduced transformer architecture and self-attention, the basis of inference.
2017-2019
ONNX was introduced at Microsoft and Meta AI. ONNX standardized Neural Networks, at this time not for LLMs but for most other forms of ML
2022
ChatGPT was publicly released, beginning the AI era.
1943
1970
2017
2019
2022

WHAT IS VERIFIABLE AI AND ZKML

zkML: (Zero-Knowledge Machine Learning) is an emerging field combining cryptographic zero-knowledge proofs (ZKPs) with machine learning algorithms. It allows a system to cryptographically prove that a machine learning model was executed correctly, all while keeping the underlying input data or the proprietary model weights completely private

VERTIFIABLE AI IS BROADER THAN JUST ZKML, IT ENCOMPASSES

- REDUCED HALLUCINATION
- RAG

ZKML: WAS COMPUTATION PERFORMED CORRECTLY?

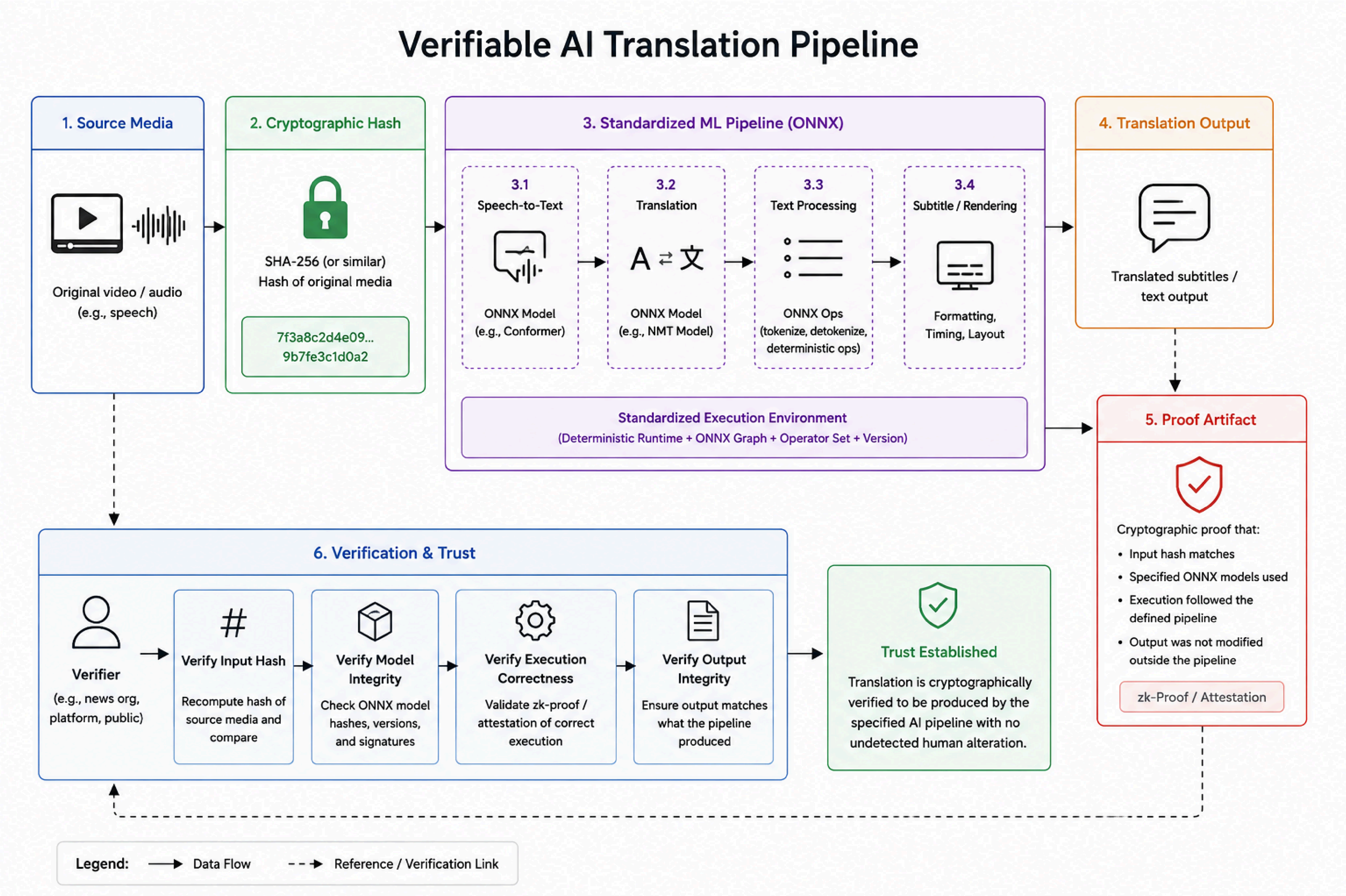
AND CAN YOU INCLUDE A CHECKABLE PROOF ARTIFACT THAT IT WAS

Use Case: Language Translation, that can be verified down to which model: a proof artifact demonstrating the output was generated by a specified model execution.

A translation system where the original media is cryptographically hashed, passed through a standardized machine learning pipeline, and accompanied by a proof artifact showing that the translated output was generated entirely by the specified model and inference process — without human alteration, censorship, or editorial tampering. Such systems could improve trust and provenance in global communication while preserving reproducibility and auditability.

other uses cases could include control systems, and autonomous systems

https://x.com/RippleXrpie/status/2055586049770811499?s=20


Verifiable AI Translation Pipeline
1. Source Media
Original video / audio (e.g., speech)
2. Cryptographic Hash
SHA-256 (or similar) Hash of original media
7f3a8c2d4e09... 9b7fe3c1d0a2
3. Standardized ML Pipeline (ONNX)
3.1 Speech-to-Text
ONNX Model (e.g., Conformer)
3.2 Translation
A ⇄ 文
ONNX Model (e.g., NMT Model)
3.3 Text Processing
ONNX Ops (tokenize, detokenize, deterministic ops)
3.4 Subtitle / Rendering
Formatting, Timing, Layout
Standardized Execution Environment
(Deterministic Runtime + ONNX Graph + Operator Set + Version)
4. Translation Output
Translated subtitles / text output
5. Proof Artifact
Cryptographic proof that:
• Input hash matches
• Specified ONNX models used
• Execution followed the defined pipeline
• Output was not modified outside the pipeline
zk-Proof / Attestation
6. Verification & Trust
Verifier
(e.g., news org, platform, public)
#
Verify Input Hash
Recompute hash of source media and compare
Verify Model Integrity
Check ONNX model hashes, versions, and signatures
Verify Execution Correctness
Validate zk-proof / attestation of correct execution
Verify Output Integrity
Ensure output matches what the pipeline produced
Trust Established
Translation is cryptographically verified to be produced by the specified AI pipeline with no undetected human alteration.
Legend:
Data Flow
Reference / Verification Link

Zero Knowledge Proofs

Recent acceleration driven by scalability and verifiable computation applications

A zero-knowledge proof (ZKP) is a cryptographic method that allows one party to prove to another that a specific statement is true, without revealing any underlying data or secrets. It mathematically verifies a claim while keeping the specific details completely hidden.

By proving you know what's in a lockbox, you can prove you know the combination without revealing the actual combination.

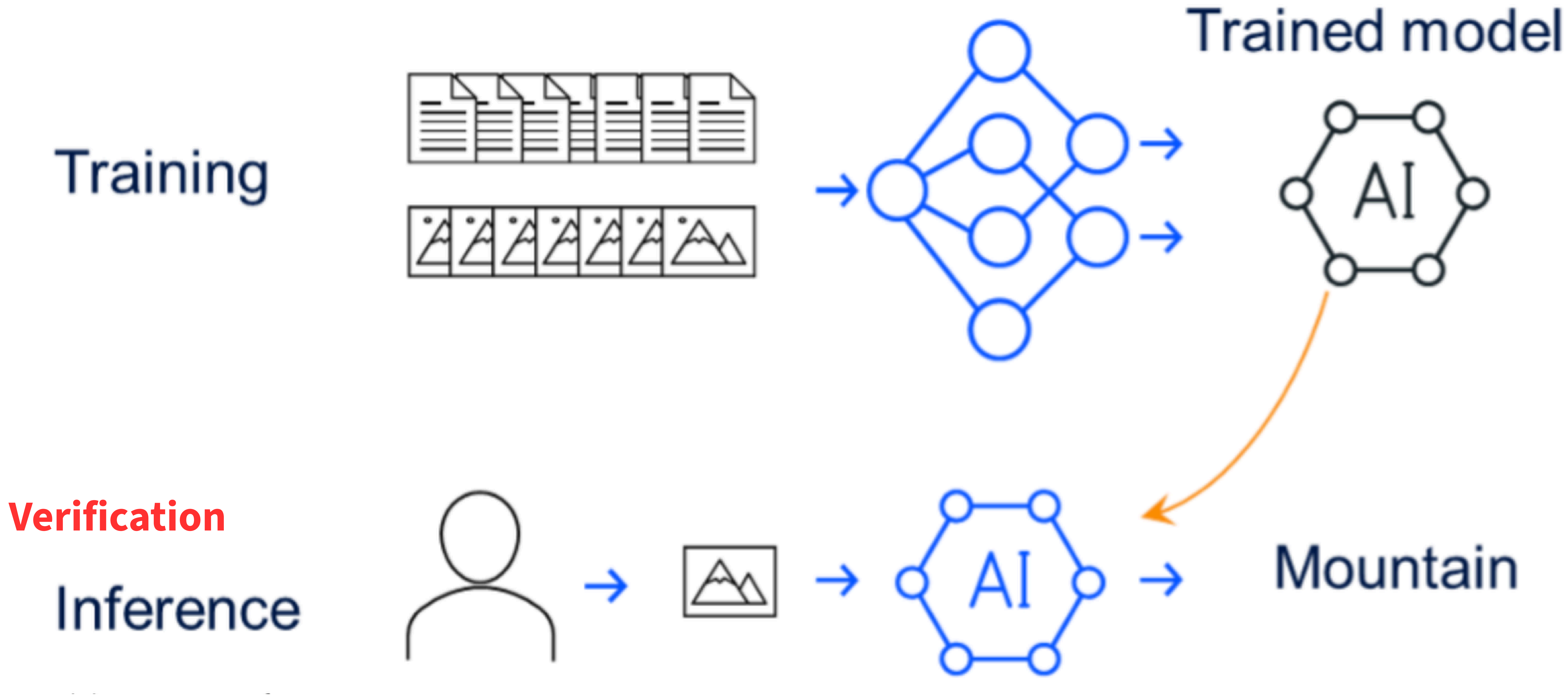
- **GKR (Goldwasser-Kalai-Rothblum)**
- **Halo2 (PLONK)**

probabilistically checkable proof / soundness error is tiny but non-zero

proof artifact

GKP was orignally developed for delegated computing

VARIAFIABLE AI PHASES



Deterministic Inference

One-to-one inference: given the same model, same prompt, and same parameters, it always produced the same output, deterministically. (including randomness)

Determinism enables reproducibility; proofs enable efficient checking.

Input

↓

ONNX Runtime

↓

Inference

↓

Proof Generator

↓

Verifier

↓

Trusted Output

THE PARELLELS OF FP AND ZKP

ZKP are a lot like type signatures, deterministic code means you can shorted computation down to shorter statements

Determism in Functional Programming

- **No random numbers**
- **No memory**
- **no external data and disk reads**
- **immutable types**

Zero Knowledge Proofs

- **execution is expected**
- **probabilistically checkable proof**
- **there's more than one valid proof per execution**

Goals: **Typechecked code is proven to work**

verified computation, proved that it executed and worked

HYPOTHESIS

Observation:

Tensor graphs behave like structured computation.

Idea:

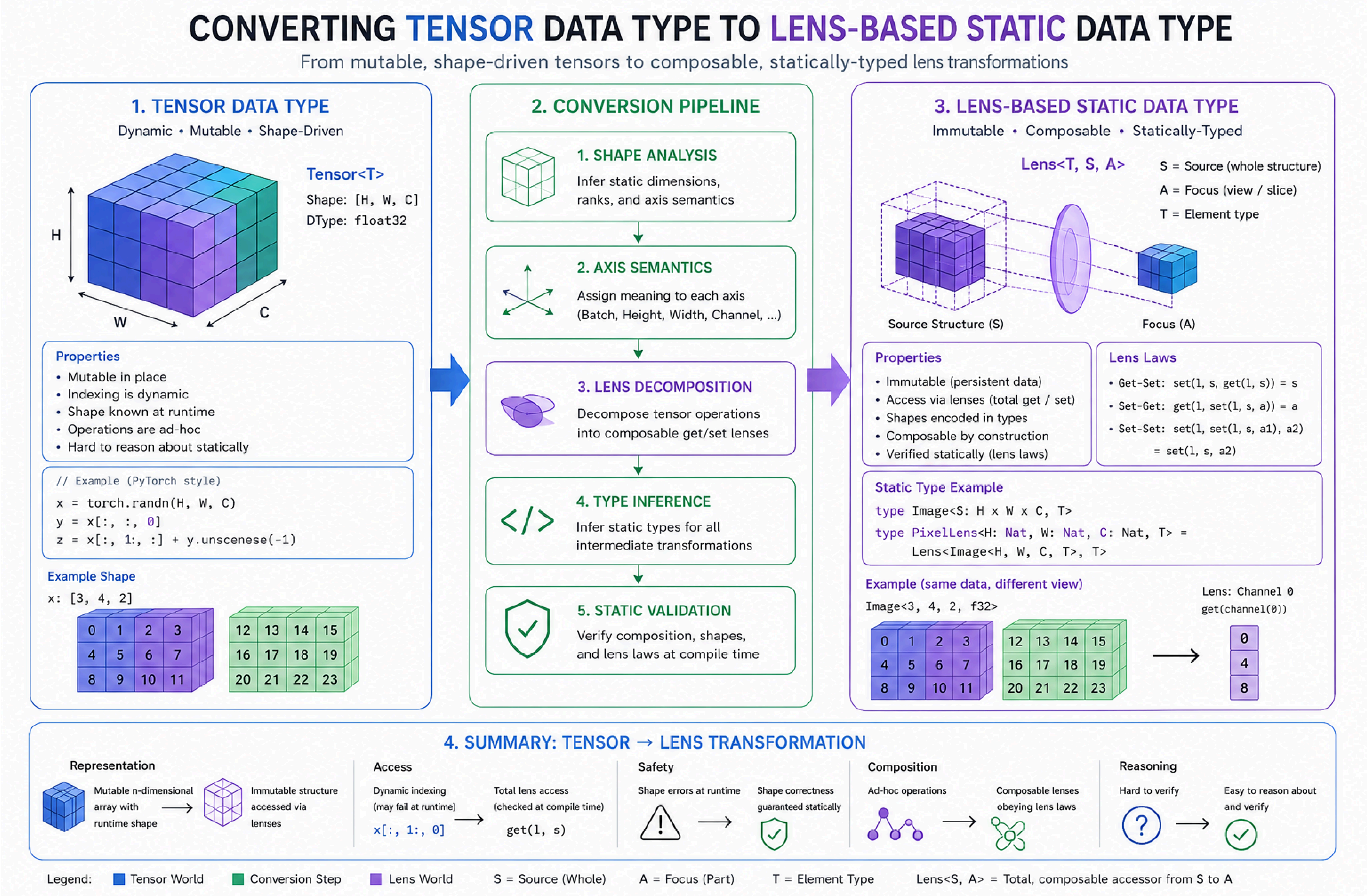
Apply typed deterministic execution.

Result:

Generate proof-friendly execution traces.

Question:

Can ONNX become a verification substrate?



CONVERTING TENSOR DATA TYPE TO LENS-BASED STATIC DATA TYPE
From mutable, shape-driven tensors to composable, statically-typed lens transformations
1. TENSOR DATA TYPE
Dynamic • Mutable • Shape-Driven
Tensor<T>
Shape: [H, W, C]
DType: float32
H
W
C
Properties
• Mutable in place
• Indexing is dynamic
• Shape known at runtime
• Operations are ad-hoc
• Hard to reason about statically
// Example (PyTorch style)
x = torch.randn(H, W, C)
y = x[:, :, 0]
z = x[:, 1:, :] + y.unscenese(-1)
Example Shape
x: [3, 4, 2]
0 1 2 3
4 5 6 7
8 9 10 11
12 13 14 15
16 17 18 19
20 21 22 23
2. CONVERSION PIPELINE
1. SHAPE ANALYSIS
Infer static dimensions, ranks, and axis semantics
2. AXIS SEMANTICS
Assign meaning to each axis (Batch, Height, Width, Channel, ...)
3. LENS DECOMPOSITION
Decompose tensor operations into composable get/set lenses
4. TYPE INFERENCE
Infer static types for all intermediate transformations
5. STATIC VALIDATION
Verify composition, shapes, and lens laws at compile time
3. LENS-BASED STATIC DATA TYPE
Immutable • Composable • Statically-Typed
Lens<T, S, A>
S = Source (whole structure)
A = Focus (view / slice)
T = Element type
Source Structure (S)
Focus (A)
Properties
• Immutable (persistent data)
• Access via lenses (total get / set)
• Shapes encoded in types
• Composable by construction
• Verified statically (lens laws)
Lens Laws
• Get-Set: set(l, s, get(l, s)) = s
• Set-Get: get(l, set(l, s, a)) = a
• Set-Set: set(l, set(l, s, a1), a2) = set(l, s, a2)
Static Type Example
type Image<S: H x W x C, T>
type PixelLens<H: Nat, W: Nat, C: Nat, T> = Lens<Image<H, W, C, T>, T>
Example (same data, different view)
Image<3, 4, 2, f32>
Lens: Channel 0
get(channel(0))
0 1 2 3
4 5 6 7
8 9 10 11
12 13 14 15
16 17 18 19
20 21 22 23
0
4
8
4. SUMMARY: TENSOR → LENS TRANSFORMATION
Representation
Mutable n-dimensional array with runtime shape
Immutable structure accessed via lenses
Access
Dynamic indexing (may fail at runtime)
x[:, 1:, 0]
Total lens access (checked at compile time)
get(l, s)
Safety
Shape errors at runtime
Shape correctness guaranteed statically
Composition
Ad-hoc operations
Composable lenses obeying lens laws
Reasoning
Hard to verify
Easy to reason about and verify
Legend: Tensor World Conversion Step Lens World S = Source (Whole) A = Focus (Part) T = Element Type Lens<S, A> = Total, composable accessor from S to A

Existing work

DeepProve: Verifiable End-to-End Large Language Model Inference (https://eprint.iacr.org/2026/1112)

EZKL: https://docs.ezkl.xyz/

zkGPT: An Efficient Non-interactive Zero-knowledge Proof Framework for LLM Inference (https://eprint.iacr.org/2025/1184.pdf)

Why ONNX?

Lens-Oriented Linearization of Tensor Inference for Zero-Knowledge Machine Learning
(link in github repo)

alphonse86-prooflens.static.hf.space/index.html

github.com/seanwestfall/zkml-presentation

ONNX June Meetup 2026

If ONNX standardized model exchange—could deterministic execution become the next layer of standardization?

sean@acceler.co.com

huggingface.co/alphonse86